

Roteiro de carga, limpeza e auditoria fiscal para arquivos ABCFarma

1. Objetivo

Este roteiro tem como objetivo:

- preparar a carga em massa de arquivos CSV fiscais em staging;
- normalizar e limpar campos antes da análise;
- executar auditoria de preços e variação de preços de medicamentos;
- automatizar o processo com uma stored procedure.

Os arquivos observados nesta pasta incluem:

- p50_abcfarma.csv
- p50_abcfarma_utf8.csv
- p50_anexo5.csv
- tabela_anexo_5_sefaz.csv
- abcfarma_completo_20260101_170957.csv

O caso de uso principal é auditoria fiscal de preços de medicamentos com foco em comparação entre períodos, laboratórios e regras de vigência.

2. Diagnóstico do arquivo

O arquivo original apresentou problemas de encoding, identificados pela sequência de bytes do início do arquivo:

- FF FE = BOM UTF-16 LE

Isso indica que o CSV foi gerado em UTF-16 e não em UTF-8. Para padronizar o trabalho, foi convertida uma cópia para UTF-8.

Conversão em PowerShell

```
$in = '.\modulo_05\roteiro_integrador_bdfisc\p50_abcfarma.csv'
$out = '.\modulo_05\roteiro_integrador_bdfisc\p50_abcfarma_utf8.csv'

$text = [System.IO.File]::ReadAllText($in, [System.Text.Encoding]::Unicode)
[System.IO.File]::WriteAllText($out, $text,
[System.Text.UTF8Encoding]::new($false))
```

Inspeção rápida do CSV no PowerShell

Antes de carregar o arquivo em massa, vale validar a estrutura e o tamanho do CSV tratado. Os comandos abaixo ajudam a confirmar o cabeçalho, a quantidade de registros e a amostra inicial.

1) Quantos campos possui a linha de cabeçalho do arquivo

```
$arquivo =
'C:\projetos\bancodedados\modulo_05\roteiro_integrador_bdfisc\p50_abcfarma_utf8.csv'

$linhaCabecalho = Get-Content $arquivo -TotalCount 1
$campos = ($linhaCabecalho -split '\|')
$campos.Count
$campos
```

O que observar: o resultado deve mostrar a quantidade correta de colunas da base ABCFarma. Como o separador é `|`, o número de campos tende a ser maior do que em arquivos com `;`.

2) Quantos registros ao todo o arquivo possui

```
$arquivo =
'C:\projetos\bancodedados\modulo_05\roteiro_integrador_bdfisc\p50_abcfarma_utf8.csv'

$registros = (Get-Content $arquivo).Count - 1
Write-Host "Total de registros em p50_abcfarma_utf8.csv: $registros"
```

3) Exibir as 10 primeiras linhas no PowerShell

```
Get-Content
```

```
'C:\projetos\bancodedados\modulo_05\roteiro_integrador_bdfisc\p50_abcfarma_utf8.csv
```

```
' -TotalCount 11
```

A armadilha: se o separador estiver incorreto, a contagem de colunas e a carga em massa podem falhar antes mesmo do **BULK INSERT**.

4. Estrutura de staging com os nomes originais do CSV

O cabeçalho de **p50_abcfarma_utf8.csv** possui 95 colunas. A tabela abaixo preserva os nomes do arquivo e a ordem original. Os colchetes são necessários nos nomes que contêm espaços, %, / , parênteses ou caracteres especiais.

```
IF SCHEMA_ID('staging') IS NULL
    EXEC ('CREATE SCHEMA staging');
GO

IF OBJECT_ID('staging.abcfarma_raw', 'U') IS NOT NULL
    DROP TABLE staging.abcfarma_raw;
GO

CREATE TABLE staging.abcfarma_raw (
    [CODIGO] NVARCHAR(200) NULL,
    [EAN] NVARCHAR(200) NULL,
    [REGISTRO ANVISA] NVARCHAR(200) NULL,
    [NCM] NVARCHAR(50) NULL,
    [CEST] NVARCHAR(50) NULL,
    [NOME] NVARCHAR(500) NULL,
    [LABORATORIO] NVARCHAR(200) NULL,
    [TIPO] NVARCHAR(50) NULL,
    [CLASSE TERAPEUTICA] NVARCHAR(500) NULL,
    [COMPOSICAO] NVARCHAR(1000) NULL,
    [TARJA] NVARCHAR(100) NULL,
    [UNIDADE DE VENDA] NVARCHAR(200) NULL,
    [CNPJ_LABORATORIO] NVARCHAR(50) NULL,
    [apresentacao] NVARCHAR(200) NULL,
    [listaPisCofins] NVARCHAR(50) NULL,
    [restrHosp] NVARCHAR(50) NULL,
    [CNPJ_DECLARANTE] NVARCHAR(50) NULL,
    [NOME_DECLARANTE] NVARCHAR(500) NULL,
    [pICMS] NVARCHAR(50) NULL,
    [vPF] NVARCHAR(50) NULL,
    [vPMC] NVARCHAR(50) NULL,
    [vPMCEmbFrac] NVARCHAR(50) NULL,
    [vPMCEmbMult] NVARCHAR(50) NULL,
    [CODIGO LABORATORIO] NVARCHAR(50) NULL,
    [LABORATÓRIO] NVARCHAR(200) NULL,
```

[DESCRIÇÃO] NVARCHAR(1000) NULL,
[COMPOSIÇÃO] NVARCHAR(1000) NULL,
[REGIME PREÇO] NVARCHAR(100) NULL,
[LISTA (LCCT)] NVARCHAR(200) NULL,
[PF 20%] NVARCHAR(50) NULL,
[PMC 20%] NVARCHAR(50) NULL,
[FRA 20%] NVARCHAR(50) NULL,
[PF 18%] NVARCHAR(50) NULL,
[PMC 18%] NVARCHAR(50) NULL,
[FRA 18%] NVARCHAR(50) NULL,
[PF 17%] NVARCHAR(50) NULL,
[PMC 17%] NVARCHAR(50) NULL,
[FRA 17%] NVARCHAR(50) NULL,
[PF 17,5%] NVARCHAR(50) NULL,
[PMC 17,5%] NVARCHAR(50) NULL,
[FRA 17,5%] NVARCHAR(50) NULL,
[PF 12%] NVARCHAR(50) NULL,
[PMC 12%] NVARCHAR(50) NULL,
[FRA 12%] NVARCHAR(50) NULL,
[PF 17% ALC] NVARCHAR(50) NULL,
[PMC 17% ALC] NVARCHAR(50) NULL,
[FRA 17% ALC] NVARCHAR(50) NULL,
[PF 17,5% ALC] NVARCHAR(50) NULL,
[PMC 17,5% ALC] NVARCHAR(50) NULL,
[FRA 17,5% ALC] NVARCHAR(50) NULL,
[PF 18% ALC] NVARCHAR(50) NULL,
[PMC 18% ALC] NVARCHAR(50) NULL,
[FRA 18% ALC] NVARCHAR(50) NULL,
[PF 0%] NVARCHAR(50) NULL,
[PMC 0%] NVARCHAR(50) NULL,
[FRA 0%] NVARCHAR(50) NULL,
[PERCENTUAL IPI] NVARCHAR(50) NULL,
[DATA VIGENCIA] NVARCHAR(50) NULL,
[NOVO_1] NVARCHAR(50) NULL,
[VARIAÇÃO DE PREÇO] NVARCHAR(200) NULL,
[REFERÊNCIA] NVARCHAR(200) NULL,
[ATC] NVARCHAR(200) NULL,
[CLASSE TERAPÊUTICA] NVARCHAR(500) NULL,
[PORTARIA 344/98] NVARCHAR(100) NULL,
[TISS/TUSS] NVARCHAR(100) NULL,
[CONFAZ 87] NVARCHAR(100) NULL,
[CAP] NVARCHAR(100) NULL,
[NOVO] NVARCHAR(50) NULL,
[GGREM] NVARCHAR(50) NULL,
[DCB] NVARCHAR(50) NULL,
[CAS] NVARCHAR(50) NULL,
[PF 22%] NVARCHAR(50) NULL,
[PMC 22%] NVARCHAR(50) NULL,
[PF 21%] NVARCHAR(50) NULL,
[PMC 21%] NVARCHAR(50) NULL,
[PF 19%] NVARCHAR(50) NULL,
[PMC 19%] NVARCHAR(50) NULL,
[PF 19% ALC] NVARCHAR(50) NULL,
[PMC 19% ALC] NVARCHAR(50) NULL,
[PF 20% ALC] NVARCHAR(50) NULL,
[PMC 20% ALC] NVARCHAR(50) NULL,
[PF 20,5%] NVARCHAR(50) NULL,
[PMC 20,5%] NVARCHAR(50) NULL,
[PF 19,5%] NVARCHAR(50) NULL,

```
[PMC 19,5%] NVARCHAR(50) NULL,  
[PF 19,5% ALC] NVARCHAR(50) NULL,  
[PMC 19,5% ALC] NVARCHAR(50) NULL,  
[PF 23%] NVARCHAR(50) NULL,  
[PMC 23%] NVARCHAR(50) NULL,  
[PF 22,5%] NVARCHAR(50) NULL,  
[PMC 22,5%] NVARCHAR(50) NULL,  
[ARQUIVO_ORIGEM] NVARCHAR(200) NULL,  
[DATA_REFERENCIA] NVARCHAR(50) NULL,  
[ANO_PASTA] NVARCHAR(20) NULL,  
[ORIGEM_ARQUIVO] NVARCHAR(50) NULL  
);  
GO
```

5. Carga em massa com BULK INSERT

O comando deve ignorar a primeira linha do cabeçalho, usar `|` como delimitador e **UTF-8** como página de código.

```
BULK INSERT staging.abcfarma_raw  
FROM 'C:\temp\p50_abcfarma_utf8.csv'  
WITH (  
    FIELDTERMINATOR = '|',  
    ROWTERMINATOR = '\n',  
    FIRSTROW = 2,  
    DATAFILETYPE = 'char',  
    CODEPAGE = '65001',  
    TABLOCK,  
    MAXERRORS = 100  
);
```

Observações

- Caso o SQL Server tenha permissão de acesso ao arquivo, a carga funciona diretamente.
 - Em caso de erro de acesso, copie o CSV para um diretório do servidor SQL ou para um compartilhamento de rede.
 - O **FIELDTERMINATOR = '|'** é obrigatório porque o arquivo usa pipe como separador.
-

6. Limpeza e normalização

Antes de inserir os dados definitivos, é importante padronizar campos numéricos e datas.

```
CREATE TABLE dbo.abcfarma_normalizado (  
    Id BIGINT IDENTITY(1,1) PRIMARY KEY,  
    CODIGO INT NULL,  
    EAN NVARCHAR(30) NULL,  
    REGISTRO_ANVISA NVARCHAR(30) NULL,  
    NOME NVARCHAR(500) NULL,  
    LABORATORIO NVARCHAR(200) NULL,  
    CLASSE_TERAPEUTICA NVARCHAR(500) NULL,  
    COMPOSICAO NVARCHAR(1000) NULL,  
    TARJA NVARCHAR(50) NULL,  
    VPF DECIMAL(18,4) NULL,  
    VPMC DECIMAL(18,4) NULL,  
    DATA_REFERENCIA DATE NULL,  
    ARQUIVO_ORIGEM NVARCHAR(200) NULL,  
    ORIGEM_ARQUIVO NVARCHAR(50) NULL  
);  
GO
```

Exemplo de transformação

```
INSERT INTO dbo.abcfarma_normalizado (  
    CODIGO,  
    EAN,  
    REGISTRO_ANVISA,  
    NOME,  
    LABORATORIO,  
    CLASSE_TERAPEUTICA,  
    COMPOSICAO,  
    TARJA,  
    VPF,  
    VPMC,  
    DATA_REFERENCIA,  
    ARQUIVO_ORIGEM,  
    ORIGEM_ARQUIVO  
)  
SELECT  
    TRY_CONVERT(INT, LTRIM(RTRIM(CODIGO))) AS CODIGO,  
    LTRIM(RTRIM(EAN)) AS EAN,  
    LTRIM(RTRIM(REGISTRO_ANVISA)) AS REGISTRO_ANVISA,  
    LTRIM(RTRIM(NOME)) AS NOME,  
    LTRIM(RTRIM(LABORATORIO)) AS LABORATORIO,  
    LTRIM(RTRIM(CLASSE_TERAPEUTICA)) AS CLASSE_TERAPEUTICA,  
    LTRIM(RTRIM(COMPOSICAO)) AS COMPOSICAO,  
    LTRIM(RTRIM(TARJA)) AS TARJA,
```

```

TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPF, '.', ''), ', ', '')) AS VPF,
TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPMC, '.', ''), ', ', '')) AS VPMC,
TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA,
LTRIM(RTRIM(ARQUIVO_ORIGEM)) AS ARQUIVO_ORIGEM,
LTRIM(RTRIM(ORIGEM_ARQUIVO)) AS ORIGEM_ARQUIVO
FROM staging.abcfarma_raw
WHERE LTRIM(RTRIM(CODIGO)) IS NOT NULL
      AND LTRIM(RTRIM(CODIGO)) <> '';

```

Funções de limpeza úteis

```

SELECT
    LTRIM(RTRIM(CODIGO)) AS CODIGO,
    REPLACE(REPLACE(VPF, '.', ''), ', ', '') AS VPF_NORMALIZADO,
    TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA
FROM staging.abcfarma_raw;

```

7. Auditoria fiscal de preços de medicamentos

O script `bdfisc.sql` não cria uma tabela histórica ABCFarma com `VPF` e `VPMC`.

Portanto, os exemplos desta seção usam os objetos fiscais existentes no script. O registro mais aderente ao caso de medicamentos é

`fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS`, gerado a partir do registro C173 da EFD. Ele contém o produto, o NCM, o lote, as datas de fabricação e validade e o preço máximo de tabela (`C173_VL_TAB_MAX`).

Para ampliar a evidência, os exemplos também usam:

- `fisc.TB_506_PR_EFD_REGISTRO_C170_ITEM_DOC_FISCAL`, com os itens escriturados e seus valores;
- `fisc.anexo5_sefaz_normalizado`, com a referência de NCM, CEST e vigência tributária;
- `dbo.ITEM_NFE`, com os itens das NF-e e os valores comerciais e tributários do documento.

7.1 Medicamentos com preço máximo de tabela ausente ou inválido

O primeiro teste identifica operações com medicamento em que o preço máximo informado no C173 está ausente ou não é positivo.

```
SELECT
    REG0_PERIODO_DECLARACAO,
    REG0_IE,
    C100_CHV_NFE,
    C100_NUM_DOC,
    C100_DT_DOC,
    REG0200_COD_ITEM,
    REG0200_DESCR_ITEM,
    REG0200_COD_NCM,
    C173_LOTE_MED,
    C173_QTD_ITEM,
    C173_VL_TAB_MAX
FROM fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS
WHERE C173_VL_TAB_MAX IS NULL
      OR C173_VL_TAB_MAX <= 0
ORDER BY C100_DT_DOC, C100_NUM_DOC;
```

Esse resultado deve ser tratado como exceção de qualidade ou de preenchimento fiscal. O valor ausente não prova, sozinho, que houve preço incorreto, mas impede a comparação com o preço máximo informado na documentação da operação.

7.2 Variação do preço máximo entre documentos

Como **TB_512** possui a data do documento e o código do item, é possível comparar o preço máximo informado em documentos consecutivos do mesmo produto.

```
CREATE OR ALTER VIEW dbo.vw_abcfarma_historico_preco_maximo
AS
    SELECT
        REG0_IE,
        REG0200_COD_ITEM,
        REG0200_DESCR_ITEM,
        C100_DT_DOC,
        C100_CHV_NFE,
        C170_NUM_ITEM,
        C173_VL_TAB_MAX,
        LAG(C173_VL_TAB_MAX) OVER (
            PARTITION BY REG0_IE, REG0200_COD_ITEM
```

```

ORDER BY C100_DT_DOC, C100_CHV_NFE, C170_NUM_ITEM
) AS PRECO_ANTERIOR
FROM fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS
WHERE C100_DT_DOC IS NOT NULL
AND C173_VL_TAB_MAX IS NOT NULL;

```

GO

A visão encapsula o cálculo do preço anterior. Assim, outras consultas e relatórios podem reutilizar o histórico sem repetir a função de janela.

```

SELECT
    REG0_IE,
    REG0200_COD_ITEM,
    REG0200_DESCR_ITEM,
    C100_DT_DOC,
    C100_CHV_NFE,
    PRECO_ANTERIOR,
    C173_VL_TAB_MAX AS PRECO_ATUAL,
    C173_VL_TAB_MAX - PRECO_ANTERIOR AS VARIACAO_PRECO,
    CASE
        WHEN ABS(C173_VL_TAB_MAX - PRECO_ANTERIOR) > 10
        THEN 'AUDITAR'
        ELSE 'NORMAL'
    END AS ALERTA
FROM dbo.vw_abcfarma_historico_preco_maximo
WHERE PRECO_ANTERIOR IS NOT NULL
AND ABS(C173_VL_TAB_MAX - PRECO_ANTERIOR) > 10
ORDER BY C100_DT_DOC DESC;

```

O limite de 10 é apenas um parâmetro didático. Em produção, ele deve ser substituído por um critério aprovado pela área fiscal, preferencialmente acompanhado de percentual de variação e intervalo entre documentos.

7.3 Medicamentos sem correspondência no Anexo 5

O código NCM escriturado no C173 pode ser comparado com a tabela normalizada do Anexo 5. O teste abaixo destaca itens sem correspondência e itens cuja referência possui CEST nulo.

```

SELECT
    m.REG0_PERIODO_DECLARACAO,
    m.C100_CHV_NFE,
    m.C100_DT_DOC,
    m.REG0200_COD_ITEM,
    m.REG0200_DESCR_ITEM,

```

```

m.REG0200_COD_NCM,
a.CEST,
a.DESCRICAO,
a.Vigencia_inicial,
a.Vigencia_final,
CASE
    WHEN a.Id IS NULL THEN 'NCM SEM REGRA NO ANEXO 5'
    WHEN a.CEST IS NULL THEN 'CEST AUSENTE NA REFERENCIA'
    ELSE 'VALIDAR ENQUADRAMENTO'
END AS ALERTA
FROM fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS AS m
LEFT JOIN fisc.anexo5_sefaz_normalizado AS a
    ON REPLACE(m.REG0200_COD_NCM, '.', '') = REPLACE(a.NCM_SH, '.', '')
WHERE a.Id IS NULL
    OR a.CEST IS NULL
ORDER BY m.C100_DT_DOC, m.REG0200_COD_NCM;

```

O código do produto não deve ser usado como substituto do CEST. A finalidade do teste é validar o NCM contra a referência oficial e, depois, revisar o enquadramento completo pela descrição e pelo regime tributário.

7.4 Regra do Anexo 5 fora da vigência da operação

A regra tributária deve ser válida na data do documento. O cruzamento abaixo usa a data do C173 e limita a referência do Anexo 5 ao intervalo de vigência.

```

SELECT
    m.REG0_PERIODO_DECLARACAO,
    m.C100_CHV_NFE,
    m.C100_DT_DOC,
    m.REG0200_COD_ITEM,
    m.REG0200_DESCR_ITEM,
    m.REG0200_COD_NCM,
    a.CEST,
    a.Vigencia_inicial,
    a.Vigencia_final,
    a.Aliq_Interna,
    a.MVA_Original
FROM fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS AS m
INNER JOIN fisc.anexo5_sefaz_normalizado AS a
    ON REPLACE(m.REG0200_COD_NCM, '.', '') = REPLACE(a.NCM_SH, '.', '')
WHERE m.C100_DT_DOC < a.Vigencia_inicial
    OR (a.Vigencia_final IS NOT NULL AND m.C100_DT_DOC > a.Vigencia_final)
ORDER BY m.C100_DT_DOC, m.REG0200_COD_NCM;

```

Essa consulta identifica uma regra encontrada pelo NCM, mas que não estava ativa na data da operação. O caso precisa ser analisado junto com o CEST, a descrição, o

regime tributário e eventuais alterações da legislação.

7.5 Validade do medicamento e data da operação

O C173 registra lote, fabricação e validade. Uma operação com data posterior à validade merece análise específica.

```
SELECT
    REG0_PERIODO_DECLARACAO,
    C100_CHV_NFE,
    C100_NUM_DOC,
    C100_DT_DOC,
    REG0200_COD_ITEM,
    REG0200_DESCR_ITEM,
    C173_LOTE_MED,
    C173_DT_FAB,
    C173_DT_VAL,
    C173_QTD_ITEM,
    C173_VL_TAB_MAX
FROM fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS
WHERE C173_DT_VAL IS NULL
      OR C173_DT_FAB IS NULL
      OR C173_DT_FAB > C173_DT_VAL
      OR C100_DT_DOC > C173_DT_VAL
ORDER BY C100_DT_DOC, C173_DT_VAL;
```

O resultado não deve ser interpretado automaticamente como venda irregular: pode haver erro de digitação, devolução, remessa ou outra situação documental. A consulta serve para selecionar os documentos que precisam de evidência complementar.

7.6 Comparação do item C170 com o medicamento do C173

O BDFISC mantém uma tabela de resultado para o C170 e outra para o C173. O cruzamento por período, chave da NF-e e número do item permite verificar se o valor do item e o preço máximo do medicamento estão coerentes.

```
SELECT
    c.REG0_PERIODO_DECLARACAO,
    c.C100_CHV_NFE,
    c.C100_NUM_DOC,
    c.C100_DT_DOC,
```

```

c.C170_NUM_ITEM,
c.REG0200_COD_ITEM,
c.REG0200_DESCR_ITEM,
c.C170_QTD,
c.C170_VL_ITEM,
m.C173_VL_TAB_MAX,
CASE
    WHEN c.C170_VL_ITEM > m.C173_VL_TAB_MAX
    THEN 'ITEM ACIMA DO PRECO MAXIMO INFORMADO'
    ELSE 'VALIDAR'
END AS ALERTA
FROM fisc.TB_506_PR_EFD_REGISTRO_C170_ITEM_DOC_FISCAL AS c
INNER JOIN fisc.TB_512_PR_EFD_REGISTRO_C173_OPERACOES_COM_MEDICAMENTOS AS m
    ON m.REG0_PERIODO_DECLARACAO = c.REG0_PERIODO_DECLARACAO
    AND m.C100_CHV_NFE = c.C100_CHV_NFE
    AND m.C170_NUM_ITEM = c.C170_NUM_ITEM
WHERE m.C173_VL_TAB_MAX IS NOT NULL
    AND c.C170_VL_ITEM > m.C173_VL_TAB_MAX
ORDER BY c.C100_DT_DOC, c.C100_NUM_DOC, c.C170_NUM_ITEM;

```

O campo **C170_VL_ITEM** é o valor total do item, enquanto **C173_VL_TAB_MAX** é o preço máximo de tabela informado no C173. Por isso, a consulta é um indicador de seleção e não uma prova definitiva de excesso: quando a quantidade for maior que uma unidade ou houver diferenças de apresentação, o auditor deve calcular também o valor unitário e conferir a unidade comercial.

7.7 Confronto com o item físico da NF-e em **dbo.ITEM_NFE**

Quando a NF-e própria estiver disponível nas tabelas operacionais, o item pode ser auditado por NCM, CEST, quantidade, valor e descrição.

```

SELECT
    i.sqnfe,
    i.tpnfe,
    i.nritemnfe,
    i.cdproduto,
    i.dsproduto,
    i.cdncm,
    i.cdcest,
    i.qtprodcom,
    i.vlprodcom,
    i.vlicms,
    i.vlicmsst
FROM dbo.ITEM_NFE AS i
WHERE i.cdncm IS NULL
    OR i.cdproduto IS NULL
    OR i.dsproduto IS NULL

```

```
OR i.qtprodcom IS NULL  
OR i.vlprodcom IS NULL;
```

Esse teste usa colunas existentes em `dbo.ITEM_NFE` e identifica dados insuficientes para uma auditoria de preço e enquadramento. O vínculo direto com `TB_512` depende da chave de nota disponível no ambiente e pode exigir uma view de relacionamento entre `sqnfe` e `C100_CHV_NFE`.

7.8 Resumo dos indicadores

- preço máximo do C173 nulo ou menor que zero;
- variação do `C173_VL_TAB_MAX` acima do limite definido;
- NCM sem correspondência no `fisc.anexo5_sefaz_normalizado`;
- regra do Anexo 5 fora da vigência do documento;
- fabricação, validade ou lote inconsistentes;
- valor do item C170 acima do preço máximo informado no C173;
- item de `dbo.ITEM_NFE` sem código, NCM, descrição, quantidade ou valor.

As tabelas `TB_506` e `TB_512` são resultados gerados pelas procedures de consulta do BDFISC. Assim, a auditoria parte da escrituração fiscal processada pelo próprio banco e usa a base ABCFarma como evidência complementar, quando o arquivo tiver sido carregado e normalizado nas tabelas da seção 6.

8. Stored procedure para automatizar o processo

```
CREATE OR ALTER PROCEDURE dbo.usp_carga_auditoria_abcfarma  
    @ArquivoCSV NVARCHAR(4000)  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    BEGIN TRY  
        IF SCHEMA_ID('staging') IS NULL  
            EXEC ('CREATE SCHEMA staging');  
  
        IF OBJECT_ID('staging.abcfarma_raw', 'U') IS NOT NULL  
            DROP TABLE staging.abcfarma_raw;  
  
        CREATE TABLE staging.abcfarma_raw (
```

CODIGO NVARCHAR(200) NULL,
EAN NVARCHAR(200) NULL,
REGISTRO_ANVISA NVARCHAR(200) NULL,
COL_04 NVARCHAR(200) NULL,
COL_05 NVARCHAR(200) NULL,
NOME NVARCHAR(500) NULL,
LABORATORIO NVARCHAR(200) NULL,
COL_08 NVARCHAR(200) NULL,
CLASSE_TERAPEUTICA NVARCHAR(500) NULL,
COMPOSICAO NVARCHAR(1000) NULL,
TARJA NVARCHAR(100) NULL,
COL_12 NVARCHAR(200) NULL,
COL_13 NVARCHAR(200) NULL,
COL_14 NVARCHAR(200) NULL,
COL_15 NVARCHAR(200) NULL,
COL_16 NVARCHAR(200) NULL,
COL_17 NVARCHAR(200) NULL,
COL_18 NVARCHAR(200) NULL,
COL_19 NVARCHAR(200) NULL,
VPF NVARCHAR(50) NULL,
VPMC NVARCHAR(50) NULL,
COL_22 NVARCHAR(200) NULL,
COL_23 NVARCHAR(200) NULL,
COL_24 NVARCHAR(200) NULL,
COL_25 NVARCHAR(200) NULL,
COL_26 NVARCHAR(200) NULL,
COL_27 NVARCHAR(200) NULL,
COL_28 NVARCHAR(200) NULL,
COL_29 NVARCHAR(200) NULL,
COL_30 NVARCHAR(200) NULL,
COL_31 NVARCHAR(200) NULL,
COL_32 NVARCHAR(200) NULL,
COL_33 NVARCHAR(200) NULL,
COL_34 NVARCHAR(200) NULL,
COL_35 NVARCHAR(200) NULL,
COL_36 NVARCHAR(200) NULL,
COL_37 NVARCHAR(200) NULL,
COL_38 NVARCHAR(200) NULL,
COL_39 NVARCHAR(200) NULL,
COL_40 NVARCHAR(200) NULL,
COL_41 NVARCHAR(200) NULL,
COL_42 NVARCHAR(200) NULL,
COL_43 NVARCHAR(200) NULL,
COL_44 NVARCHAR(200) NULL,
COL_45 NVARCHAR(200) NULL,
COL_46 NVARCHAR(200) NULL,
COL_47 NVARCHAR(200) NULL,
COL_48 NVARCHAR(200) NULL,
COL_49 NVARCHAR(200) NULL,
COL_50 NVARCHAR(200) NULL,
COL_51 NVARCHAR(200) NULL,
COL_52 NVARCHAR(200) NULL,
COL_53 NVARCHAR(200) NULL,
COL_54 NVARCHAR(200) NULL,
COL_55 NVARCHAR(200) NULL,
COL_56 NVARCHAR(200) NULL,
COL_57 NVARCHAR(200) NULL,
COL_58 NVARCHAR(200) NULL,
COL_59 NVARCHAR(200) NULL,

```

COL_60 NVARCHAR(200) NULL,
COL_61 NVARCHAR(200) NULL,
COL_62 NVARCHAR(200) NULL,
COL_63 NVARCHAR(200) NULL,
COL_64 NVARCHAR(200) NULL,
COL_65 NVARCHAR(200) NULL,
COL_66 NVARCHAR(200) NULL,
COL_67 NVARCHAR(200) NULL,
COL_68 NVARCHAR(200) NULL,
COL_69 NVARCHAR(200) NULL,
COL_70 NVARCHAR(200) NULL,
COL_71 NVARCHAR(200) NULL,
COL_72 NVARCHAR(200) NULL,
COL_73 NVARCHAR(200) NULL,
COL_74 NVARCHAR(200) NULL,
COL_75 NVARCHAR(200) NULL,
COL_76 NVARCHAR(200) NULL,
COL_77 NVARCHAR(200) NULL,
COL_78 NVARCHAR(200) NULL,
COL_79 NVARCHAR(200) NULL,
COL_80 NVARCHAR(200) NULL,
COL_81 NVARCHAR(200) NULL,
COL_82 NVARCHAR(200) NULL,
COL_83 NVARCHAR(200) NULL,
COL_84 NVARCHAR(200) NULL,
COL_85 NVARCHAR(200) NULL,
COL_86 NVARCHAR(200) NULL,
COL_87 NVARCHAR(200) NULL,
COL_88 NVARCHAR(200) NULL,
COL_89 NVARCHAR(200) NULL,
COL_90 NVARCHAR(200) NULL,
COL_91 NVARCHAR(200) NULL,
ARQUIVO_ORIGEM NVARCHAR(200) NULL,
DATA_REFERENCIA NVARCHAR(50) NULL,
ANO_PASTA NVARCHAR(20) NULL,
ORIGEM_ARQUIVO NVARCHAR(50) NULL
);

DECLARE @SQLBulkInsert NVARCHAR(MAX) =
N'BULK INSERT staging.abcfarma_raw
FROM ''' + REPLACE(@ArquivoCSV, '''', ''''') + N'''
WITH (
    FIELDTERMINATOR = '|' ,
    ROWTERMINATOR = '\n',
    FIRSTROW = 2,
    DATAFILETYPE = 'char',
    CODEPAGE = '65001',
    TABLOCK,
    MAXERRORS = 100
);';

EXEC sys.sp_executesql @SQLBulkInsert;

;WITH base AS (
    SELECT
        TRY_CONVERT(INT, LTRIM(RTRIM(CODIGO))) AS CODIGO,
        LTRIM(RTRIM(EAN)) AS EAN,
        LTRIM(RTRIM([REGISTRO ANVISA])) AS REGISTRO_ANVISA,
        LTRIM(RTRIM(NOME)) AS NOME,

```

```

        LTRIM(RTRIM(LABORATORIO)) AS LABORATORIO,
        LTRIM(RTRIM([CLASSE TERAPEUTICA])) AS CLASSE_TERAPEUTICA,
        LTRIM(RTRIM(COMPOSICAO)) AS COMPOSICAO,
        LTRIM(RTRIM([TARJA])) AS TARJA,
        TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPF, '.', ''), ',', '')) AS VPF,
        TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPMC, '.', ''), ',', '')) AS VPMC,
        TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA,
        LTRIM(RTRIM(ARQUIVO_ORIGEM)) AS ARQUIVO_ORIGEM,
        LTRIM(RTRIM(ORIGEM_ARQUIVO)) AS ORIGEM_ARQUIVO
    FROM staging.abcfarma_raw
    WHERE LTRIM(RTRIM(CODIGO)) IS NOT NULL
        AND LTRIM(RTRIM(CODIGO)) <> ''
)
INSERT INTO dbo.abcfarma_normalizado (
    CODIGO,
    EAN,
    REGISTRO_ANVISA,
    NOME,
    LABORATORIO,
    CLASSE_TERAPEUTICA,
    COMPOSICAO,
    TARJA,
    VPF,
    VPMC,
    DATA_REFERENCIA,
    ARQUIVO_ORIGEM,
    ORIGEM_ARQUIVO
)
SELECT
    CODIGO,
    EAN,
    REGISTRO_ANVISA,
    NOME,
    LABORATORIO,
    CLASSE_TERAPEUTICA,
    COMPOSICAO,
    TARJA,
    VPF,
    VPMC,
    DATA_REFERENCIA,
    ARQUIVO_ORIGEM,
    ORIGEM_ARQUIVO
FROM base;

;WITH base AS (
    SELECT
        CODIGO,
        LABORATORIO,
        DATA_REFERENCIA,
        VPF,
        VPMC,
        LAG(VPF) OVER (PARTITION BY CODIGO, LABORATORIO ORDER BY
DATA_REFERENCIA) AS VPF_ANTERIOR,
        LAG(VPMC) OVER (PARTITION BY CODIGO, LABORATORIO ORDER BY
DATA_REFERENCIA) AS VPMC_ANTERIOR
    FROM dbo.abcfarma_normalizado
)

```

```

INSERT INTO dbo.auditoria_abcfarma (
    CODIGO,
    LABORATORIO,
    DATA_REFERENCIA,
    VPF,
    VPMC,
    VPF_ANTERIOR,
    VPMC_ANTERIOR,
    VARIACAO_VPF,
    VARIACAO_VPMC,
    FLAG_DIVERGENCIA
)
SELECT
    CODIGO,
    LABORATORIO,
    DATA_REFERENCIA,
    VPF,
    VPMC,
    VPF_ANTERIOR,
    VPMC_ANTERIOR,
    ISNULL(VPF - VPF_ANTERIOR, 0),
    ISNULL(VPMC - VPMC_ANTERIOR, 0),
    CASE
        WHEN ABS(CAST((VPF - ISNULL(VPF_ANTERIOR, VPF)) AS DECIMAL(18,4)))
> 10
            OR ABS(CAST((VPMC - ISNULL(VPMC_ANTERIOR, VPMC)) AS
DECIMAL(18,4))) > 10
        THEN 1 ELSE 0
    END
FROM base
WHERE DATA_REFERENCIA IS NOT NULL;

SELECT 'Carga e auditoria concluídas com sucesso.' AS Resultado;

END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumeroErro,
        ERROR_MESSAGE() AS MensagemErro;
    THROW;
END CATCH
END;
GO

```

Execução

```

EXEC dbo.usp_carga_auditoria_abcfarma
    @ArquivoCSV = 'C:\dados\staging\p50_abcfarma_utf8.csv';

```

9. Checklist de execução

- ☐ confirmar encoding do arquivo original;
- ☐ converter para UTF-8 quando necessário;
- ☐ validar separador |;
- ☐ criar schema `staging`;
- ☐ criar tabela de staging;
- ☐ executar `BULK INSERT`;
- ☐ limpar e converter campos numéricos;
- ☐ carregar tabela normalizada;
- ☐ gerar auditoria de variação de preços;
- ☐ automatizar com stored procedure.

10. Conclusão

Este fluxo reproduz um cenário típico de auditoria fiscal com dados externos em arquivo CSV: carga em massa, etapa de staging, limpeza, análise e materialização de resultados. Com a base padronizada, o auditor pode identificar anomalias de preço, comparar períodos e priorizar produtos com risco fiscal.

Se for necessário, o próximo passo pode ser:

- adaptação da rotina para SQL Server real do ambiente;
- uso dos arquivos `p50_anexo5.csv` e `tabela_anexo_5_sefaz.csv` como complemento da auditoria;
- geração de relatórios por laboratório, NCM ou faixa de preço;
- criação de view consolidada para uso em dashboard de auditoria.